



Research Description of the Accelera Parallelizer and Pipeline Modules

Mohamed Ashraf Nesma Osama Mazen Adel Mohamed Khater
ID: 9220658 ID: 9220912 ID: 9220625 ID: 9220713

Supervisor: Dr. Salma Abdel monem

April 17, 2026

Abstract

Accelera is a hybrid Python/C++ machine learning framework designed to combine Python's modelling productivity with C++ execution infrastructure. The project contains graph-based machine learning pipelines, parallel branch execution, HTML reporting, AutoML preprocessing utilities, dataset retrieval, deployment support, benchmarking infrastructure, and a Linux-only C/C++ loop parallelizer. This report is organized around five project modules: `accelera_pipe`, the code parallelizer, AutoML, deployment, and the benchmark platform. The `accelera_pipe` module constructs and executes directed machine learning workflows using a Python API over a C++ graph backend. The code parallelizer analyzes source loops, extracts features, classifies loops, and injects OpenMP pragmas. The remaining module sections are reserved for future expansion.

Contents

1	Project Introduction	3
2	Accelera Pipe Module	3
2.1	Module Location and Responsibility	3
2.2	Graph Backend	3
2.3	Pipeline Builder API	4
2.4	Execution Semantics	4
2.5	Preprocessing and Model Fitting	4
2.6	Prediction Nodes	4
2.7	Metric Nodes	4
2.8	Branching and Merging	5
2.9	Executed Graphs	5
2.10	Reporting	5
2.11	Empirical Comparison with sklearn	6
2.12	Implemented Pipeline Optimizations	7
2.13	How to Use the Accelera Pipe Module	9

3	Code Parallelizer Module	10
3.1	Module Location and Responsibility	10
3.2	Native Loop Extraction	11
3.3	Feature Extraction	11
3.4	Vectorization for the Classifier	11
3.5	Loop Class Resolution	12
3.6	OpenMP Pragma Generation	12
3.7	Generator Model Trial and Rule-Based Design Rationale	12
3.8	Python-to-C++ Converter	13
3.9	Limitations	14
3.10	How to Use the Parallelizer	14
4	AutoML Module	15
4.1	AutoPreprocessing	15
4.1.1	Supported Dataset Types	15
4.1.2	Supported Tasks	15
4.1.3	Tabular Dataset	15
4.1.4	Text Dataset	21
5	Deployment Module	24
6	Benchmark Platform Module	24
7	Module-Level View	24
8	Conclusion	24

1 Project Introduction

Accelera targets experimental and production-oriented machine learning workflows where a user wants to express a pipeline in Python while delegating graph execution and branch scheduling to a compiled backend. Its main design idea is to represent preprocessing, model training, prediction, merging, and metric computation as graph nodes. Independent branches can then be evaluated in parallel, making it convenient to compare alternative preprocessors, estimators, and metrics in a single run.

The repository also includes a code parallelizer that operates outside the ML pipeline. That subsystem parses C/C++ source code through Clang AST bindings, extracts loop fragments, computes structural and dependency-related features, consults a remote classifier service, and inserts OpenMP `parallel` for pragmas where the loop appears safe or useful to parallelize. If the input file is Python, a small AST-based converter first emits equivalent C++ for a limited subset of Python syntax before loop analysis begins.

2 Accelera Pipe Module

2.1 Module Location and Responsibility

The `accelera_pipe` module is implemented under `accelera/src/accelera_pipe/`. It provides the Python API for building graph pipelines and report objects. Its core files are:

- `core/pipeline_base.py`: wraps the C++ `graph.Graph` object and exposes shared configuration methods.
- `core/pipeline.py`: public builder API for preprocessing, model, prediction, metric, merge, and branch nodes.
- `core/executed_graph.py`: reusable executed graph wrapper.
- `core/metric.py`: abstract metric interface.
- `wrappers/supervised_metric.py` and `wrappers/unsupervised_metric.py`: concrete metric adapters.
- `core/report_base.py` and wrapper display classes: HTML report generation for pipeline graphs and metrics.

2.2 Graph Backend

`PipelineBase` imports the native `graph` module. If this module is missing, it raises an import error that instructs the user to build the C++ bindings and make them available in `PYTHONPATH`. When initialized, the base class creates or reuses a `graph.Graph` instance and enables parallel execution by default:

```
self.__graph = _graph if _graph is not None else graph.Graph()
self.__graph.enableParallelExecution(True)
```

The node type mapping bridges Python method names to C++ enum values: `PREPROCESS`, `MODEL`, `PREDICT`, `METRIC`, and `MERGE`.

2.3 Pipeline Builder API

Pipeline is the main user-facing class. Each method either appends a node to the graph or, when `branch=True`, returns a lightweight `Node` object that can be supplied to `branch()`.

Table 1: Main Pipeline methods.

Method	Purpose	Important parameters
<code>preprocess</code>	Add a transformation step.	<code>name</code> , <code>func</code> , <code>branch</code> .
<code>model</code>	Add a trainable model.	<code>name</code> , <code>model</code> , <code>branch</code> .
<code>predict</code>	Add prediction over test data.	<code>test_data</code> , <code>output_func</code> , <code>positive_class</code> .
<code>metric</code>	Add a supervised or unsupervised metric.	<code>metric_name</code> , <code>y_true</code> , <code>plot_func</code> , <code>labels</code> , <code>headers</code> , extra metric parameters.
<code>merge</code>	Combine branch predictions.	<code>strategy</code> , currently defaulting to <code>hard_voting</code> .
<code>branch</code>	Split the graph into alternative paths.	A branch name and one or more <code>Node</code> objects created with <code>branch=True</code> .

2.4 Execution Semantics

Calling a Pipeline executes the underlying graph:

```
predictions, executed_graph = pipe(X, y, select_strategy="all")
```

The C++ backend returns an executed graph plus result values. The Python layer wraps the first returned item in `ExecutedGraph`; all remaining returned items are predictions or metric outputs. The selection strategy can be `all`, `max`, `min`, or a user-provided `custom_strategy`. Tests in the module show that these strategies are used to select paths according to metric outputs.

2.5 Preprocessing and Model Fitting

Preprocess nodes store the transformation object or function together with `execute_fit`, a helper that determines whether a component should be fit with `X` only or with both `X` and `y`. This lets the pipeline handle unsupervised transformers such as scalers and supervised transformers such as feature selectors. Model nodes similarly store the model object and fitting helper, allowing sklearn-compatible estimators and custom Accelerate estimators to participate in the graph.

2.6 Prediction Nodes

`predict(name, test_data, output_func="predict", positive_class=-1)` records the data on which the trained model should be evaluated. The `output_func` parameter can request outputs such as `predict` or `predict_proba`. When probabilities are used, `positive_class` can select one probability column.

2.7 Metric Nodes

The pipeline maps metric names to functions from `sklearn.metrics`. Then the metric-class helper inspects the metric function signature:

- A metric with true labels and predicted labels/scores becomes a **SupervisedMetric**.
- A metric with **X** and **labels** becomes an **UnSupervisedMetric**.
- Any incompatible signature is rejected.

SupervisedMetric.execute validates that **y_pred** and **y_true** have the same number of samples, then calls the sklearn metric. **UnSupervisedMetric.execute** validates predictions against the stored input **X**. Both return a dictionary containing the metric name, result, plot function, label names, and header names.

2.8 Branching and Merging

branch() accepts one or more branch node objects. Each branch can be a single node or an iterable of nodes. The method validates that all branch members are **Node** instances and rejects merge nodes inside branches because merge-in-branch behavior is not implemented yet. The validated branch objects, node types, and node names are passed to the C++ graph through **split()**.

Merge nodes combine multiple branch outputs. The default strategy is **hard_voting**. The test suite covers hard-voting merges, merges after preprocessing branches, probability prediction merges, single-model merges, and reuse of executed graphs after merging.

2.9 Executed Graphs

ExecutedGraph wraps a graph that has already been fitted or selected. Calling it with new **X** reuses the fitted pipeline. If **y_true** is provided, metrics are temporarily enabled for that call and disabled afterward.

Executed graphs can now be persisted as a single pickle file. Calling **save()** with no arguments creates **pipeline.pkl** in the current working directory. The user may also pass an explicit file path such as **"my_pipeline.pkl"**. Loading is performed through **ExecutedGraph.load()**, which reconstructs the native C++ graph and restores the fitted Python objects stored in the graph nodes. The loaded graph keeps the executed-state flag, so applying data to it performs inference directly rather than fitting the models again.

```
predictions, executed_graph = pipeline(X_train, y_train)

executed_graph.save()           # writes pipeline.pkl
loaded_graph = ExecutedGraph.load() # reads pipeline.pkl

test_predictions = loaded_graph(X_test)
```

The implementation uses a pybind pickle bridge for the C++ **Graph** object. During saving, the graph is converted into a Python dictionary containing node metadata, source-edge indices, selected-path flags, fitted preprocessors, and fitted model state. During loading, this dictionary is used to rebuild the graph topology and restore each node. This design stores the pipeline in one file, but it requires the Python objects inside the pipeline to be pickle-compatible; standard sklearn transformers and estimators satisfy this requirement, while lambdas and some local custom functions may not.

2.10 Reporting

The reporting layer creates HTML reports rather than PDF reports. The base class **ReportBase** defines the HTML shell and utility functions. **GraphReport** reads serialized pipeline XML, builds a Graphviz diagram, colors selected path nodes red and other nodes blue, lists branches, and then delegates metric rendering to **GraphPipelineReport**.

The graph-pipeline report groups metric records by metric name and chooses a display wrapper according to the result type:

- scalar values: `DisplaySingleNumber`;
- one-dimensional arrays: `DisplayArraySingle`;
- multi-dimensional arrays: `DisplayMultiArray` or `DisplayFigure`;
- dictionaries: `DisplayDict`;
- strings: `DisplayString`;
- tuples: `DisplayTupleNotCurve` or `DisplayFigure`.

`ModelReport` reuses this metric rendering and can optionally plot training history when a dictionary of history arrays is provided.

2.11 Empirical Comparison with sklearn

To evaluate the execution behavior of `accelera_pipe`, a controlled comparison was performed against two sklearn baselines. The first baseline uses a direct manual sklearn loop, where each candidate scaler and model is cloned, fitted, evaluated on validation data, and then the selected candidate is evaluated on test data. The second baseline uses `sklearn.pipeline` with the same candidate pairs. The `Accelerate` version expresses the same candidate space as two graph branches: two preprocessing alternatives (`StandardScaler` and `MinMaxScaler`) followed by two model alternatives (`LogisticRegression` and `SVC`). Therefore, all systems evaluate four candidate pipelines and select the path with the highest validation accuracy.

The experiment uses synthetic four-class classification data with 25 features. Each dataset is split into 60% training, 20% validation, and 20% test data. The validation split is used only for candidate selection, while the test split is used to report final accuracy. Table 2 shows that all three implementations selected an equivalent best candidate and obtained the same test accuracy in every run. For small inputs, the native graph overhead dominates; for larger inputs, the parallel branch execution of `accelera_pipe` reduces latency compared with both sklearn baselines. The memory measurements also show that `accelera_pipe` currently pays a larger memory cost because multiple graph branches and fitted node states can be alive at the same time.

Table 2: Fair comparison between manual sklearn, sklearn Pipeline, and `accelera_pipe`. Time is measured in seconds and RSS is the process resident-memory delta in MB.

Samples	Train	Val	Test	sklearn Time	sklearn RSS	sklearn Pipe Time	sklearn Pipe RSS	Accelerate Time	Accelerate RSS
1,000	600	200	200	0.06	1.62	0.05	0.16	0.34	19.32
5,000	3,000	1,000	1,000	2.47	3.62	2.12	1.09	2.02	21.62
10,000	6,000	2,000	2,000	2.64	11.25	2.67	2.19	2.10	28.43
50,000	30,000	10,000	10,000	23.96	126.95	23.25	9.75	14.81	209.25
100,000	60,000	20,000	20,000	77.79	139.70	77.10	47.65	67.04	300.93

Table 3: Accuracy and selected candidate behavior. The test-accuracy delta is computed as `accelera_pipe` minus manual sklearn.

Samples	sklearn Best Candidate	Accelerate Selected Candidate	Validation Accuracy	Test Accuracy	Test Accuracy Delta
1,000	MinMaxScaler → SVC	MinMaxScaler → SVC	0.7900	0.8250	+0.0000
5,000	MinMaxScaler → SVC	MinMaxScaler → SVC	0.9160	0.9100	+0.0000
10,000	MinMaxScaler → SVC	MinMaxScaler → SVC	0.9385	0.9260	+0.0000
50,000	MinMaxScaler → SVC	MinMaxScaler → SVC	0.9565	0.9598	+0.0000
100,000	StandardScaler → SVC	StandardScaler → SVC	0.9675	0.9709	+0.0000

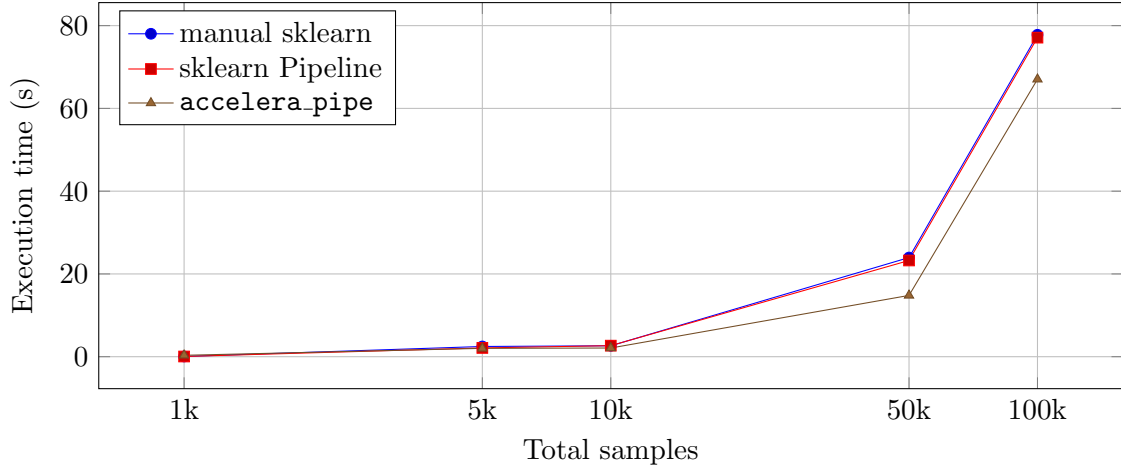


Figure 1: Latency scaling for the same four-candidate search space.

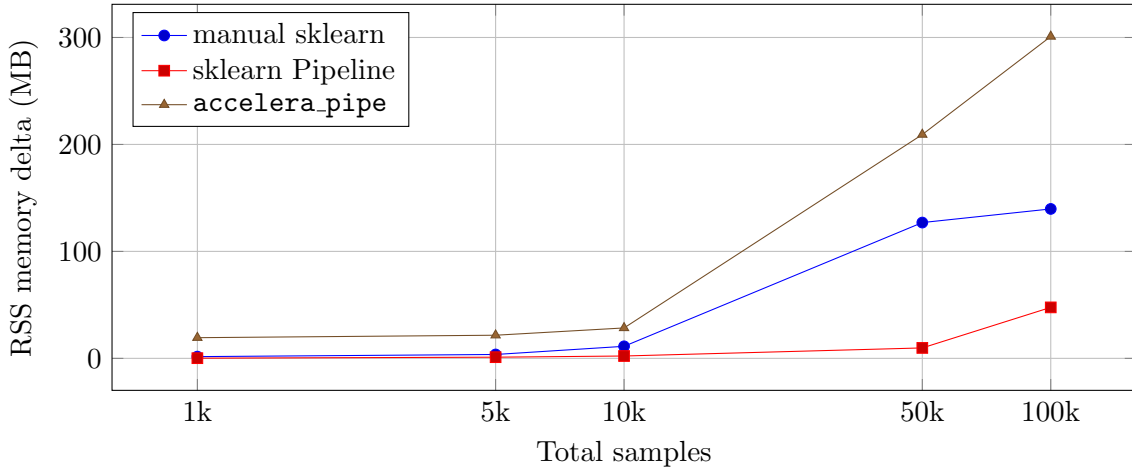


Figure 2: RSS memory delta for the same benchmark runs.

2.12 Implemented Pipeline Optimizations

Several optimizations were added to reduce unnecessary work and memory retention in the native graph backend. These changes target the main cost of branch-based pipeline search: multiple candidate paths can hold similar intermediate arrays, fitted states, and prediction outputs at the same time.

Optional cache execution. Caching for preprocess and model nodes was made opt-in. The public Python API keeps the parameter `cache=False` by default for both `preprocess()` and `model()`. When caching is disabled, the C++ nodes skip the expensive `joblib.hash()`, `joblib.load()`, and `joblib.dump()` path. This reduces latency and memory pressure in normal search runs, while preserving cache support for repeated experiments where the user explicitly enables it.

Consumer-count based intermediate release. The graph backend now computes how many downstream nodes still consume each node output. After a node finishes, the consumer count of its source nodes is decremented. When the count reaches zero, temporary data for releasable nodes is cleared. This is especially important for preprocessing outputs, because a

transformed training array may be needed by multiple model branches but becomes dead once the last model has finished fitting.

Input training-data release. The original input node stores the raw training `X` and `y`. After all downstream training consumers have executed, these references are cleared as well. This prevents the graph from retaining the original training arrays after fitted preprocessors and models no longer require them. The release is disabled for executed graphs during inference, because the executed graph input must remain available while prediction nodes apply the fitted preprocessing chain to new test data.

Separated output-container creation from input copying. Originally, the internal flag `should_create_new_data` controlled two different behaviors: creating a fresh branch output container and copying `X/y` before preprocessing. These meanings were separated into two concepts. A first preprocessing node in a branch still creates its own output dictionary, but the decision to defensively copy the input is controlled independently. This avoids corrupting branch storage semantics while allowing safe copy reduction.

Guarded defensive-copy reduction for preprocessing nodes. The graph separates branch-output creation from input-copy decisions through the `shouldCopyPreprocessInput()` helper. A return value of `true` means that the preprocessing node receives defensive copies of `X` and `y`; a return value of `false` means that the node can safely read the existing branch input without copying it first. The helper first rejects non-preprocessing nodes, then inspects the Python preprocessing object stored under the `func` key. Non-sklearn callables remain conservative and are copied, because an arbitrary user function may mutate its input in place.

For sklearn transformer instances, copying is disabled only when the transformer is not configured for in-place mutation. The implementation calls `get_params(deep=False)` when available and checks the sklearn `copy` parameter. If `copy=False`, the graph keeps the defensive copy because the transformer is explicitly allowed to mutate its input. If the parameter is absent or inspection succeeds without finding an unsafe configuration, the copy is skipped. If inspection raises a Python error, the backend clears the error and falls back to copying. This design reduces memory for common sklearn preprocessors such as `StandardScaler` and `MinMaxScaler` under their default settings, while preserving isolation for custom callables and sklearn transformers configured with `copy=False`.

Executed-graph data retention reduction. Executed graph cloning keeps the fitted model state needed for future prediction, but avoids carrying unnecessary training intermediates such as raw training arrays and preprocessing output dictionaries. The selected executable path therefore keeps fitted preprocessors and fitted models as reusable state, while temporary training data can be released.

Duplicate first-branch-node sharing. The split operation now deduplicates identical first nodes created from the same split source. For example, if a branch is declared as:

```
pipe.branch(  
    "preprocessing",  
    pipe.preprocess("a", MinMaxScaler(), branch=True),  
    pipe.preprocess("b", StandardScaler(), branch=True),  
    pipe.preprocess("c", MinMaxScaler(), branch=True),  
)
```

the graph creates one `MinMaxScaler` node and one `StandardScaler` node rather than two equivalent `MinMaxScaler` nodes. The signature used for this sharing depends on the node type

and Python object representation, not on the user-facing node name, so identical operations with different names can still be shared.

The optimization is intentionally restricted to first branch nodes. For list branches, only the first item is eligible for sharing:

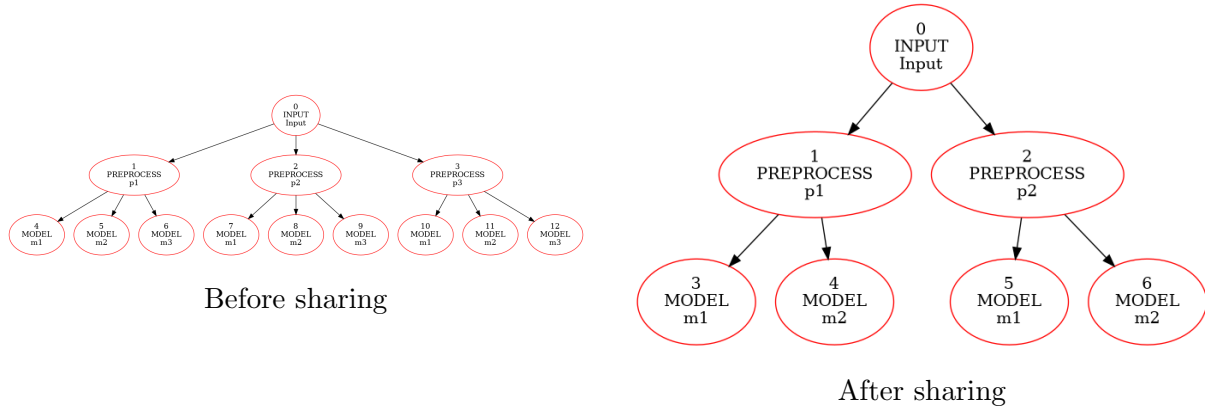


Figure 3: Branch graph before and after duplicate first-node sharing.

Later nodes are not merged, even if they have the same operation name or object representation, because their inputs may differ. For example, `[p1, p2, p3]` and `[p4, p2, p3]` keep separate `p2` nodes because one `p2` receives data from `p1`, while the other receives data from `p4`. This preserves correctness while removing obvious duplicate work at branch entry points.

Fair benchmark protocol. The comparison script was adjusted so manual `sklearn`, `sklearn Pipeline`, and `accelera_pipe` follow the same protocol: train and validate all four candidate paths, select the best validation path, and evaluate the test split only once for the selected path. This makes latency comparisons more meaningful by avoiding extra test predictions in the `sklearn` baselines.

2.13 How to Use the Accelera Pipe Module

A minimal classification pipeline is:

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

from accelera.src.accelera_pipe.core.pipeline import Pipeline

X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
X_test, y_test = X[:100], y[:100]

pipe = Pipeline()
pipe.preprocess("scale", StandardScaler())
pipe.model("logreg", LogisticRegression(max_iter=1000))
pipe.predict("predict", test_data=X_test)
pipe.metric("accuracy", "accuracy_score", y_true=y_test)

results, executed_graph = pipe(X, y)
print(results)
```

A branching experiment compares preprocessors and models:

```
from sklearn.ensemble import RandomForestClassifier
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import MinMaxScaler, StandardScaler

pipe = Pipeline()
pipe.branch(
    "preprocessing",
    pipe.preprocess("standard", StandardScaler(), branch=True),
    pipe.preprocess("minmax", MinMaxScaler(), branch=True),
)
pipe.branch(
    "models",
    pipe.model("logreg", LogisticRegression(max_iter=1000), branch=True),
    pipe.model("forest", RandomForestClassifier(n_estimators=50), branch=True),
)
pipe.predict("predict", test_data=X_test)
pipe.metric("accuracy", "accuracy_score", y_true=y_test)

results, executed_graph = pipe(X, y, select_strategy="max")
best_result = executed_graph(X_test, y_test)

```

To generate a graph report:

```

from accelera.src.accelera_pipe.wrappers.graph_report import GraphReport
from accelera.src.utils.accelera_utils import serialize

results, executed_graph = pipe(X, y, select_strategy="max")
serialize(pipe, "pipeline.xml")

report = GraphReport("pipeline_report", "pipeline.xml", results)
report.execute()

```

3 Code Parallelizer Module

3.1 Module Location and Responsibility

The parallelizer is implemented mainly in `accelera/src/utils/parallelizer.py`. Its Python-to-C++ helper is in `accelera/src/utils/py2cpp_converter.py`. Native loop extraction is provided by the pybind module `code_parallelizer_utils`, which is built from C++ sources under `src/ast/` and `src/utils/code_parallelizer_utils.cpp`. The CMake configuration enables these bindings only on Linux with LLVM/Clang available.

Conceptually, the module implements the following pipeline:

1. Read a C, C++, or supported Python file.
2. Convert Python input to C++ when the file extension is `.py`.
3. Extract loops using Clang-backed native bindings.
4. Cache extracted loop metadata in `.accelera_cache`.
5. Compute a 40-dimensional feature vector for each `for` loop.
6. Send the vector to the classifier endpoint configured by `ACCELERA_CLASSIFIER_ENDPOINT`.
7. Resolve the final loop class with local heuristics.
8. Generate OpenMP pragmas and replace the original loop text.

9. Write `parallelized_<name>.c` beside the input file and format it with `clang-format` when available.

3.2 Native Loop Extraction

The public loop extraction helpers wrap the native `code_parallelizer_utils` bindings. The default Clang argument is `-std=c++17`. The expected extracted loop records include at least the loop source code, loop type, start line, and end line. Although the C++ AST layer can identify several loop kinds, the Python insertion logic currently parallelizes only loops whose extracted type is `for`.

3.3 Feature Extraction

The function `extract_features(code)` performs lightweight static analysis using regular expressions after comment removal. The extracted features are intentionally pragmatic: they describe loop structure, reduction patterns, memory access shape, control flow, and rough computational intensity. Important feature groups include:

- **Reduction features:** detection of scalar `+=`, `x = x + ...`, `*=`, `x = x * ...`, and textual `min/max` usage.
- **Dependency features:** loop-carried dependency patterns such as `a[i - 1]`, read-after-write flags, write-after-read flags, and dependency distance.
- **Memory features:** array reads, array writes, indirect indexing, number of memory operations, and hashed array-name embeddings.
- **Control-flow features:** branch count, early exits via `break`, `continue`, or `return`, and function call counts.
- **Shape features:** constant loop bounds, estimated iteration count, consecutive nested loop depth, and nested brace depth.
- **Vectorization hint:** a boolean flag that becomes true when the loop has no branches, no function calls, no loop-carried dependency, and no indirect access.

3.4 Vectorization for the Classifier

`vectorize_features(features)` maps the feature dictionary to a 40-element `numpy.float32` vector. Boolean features occupy positions 0–16. Counts and normalized scalar descriptors occupy later positions. Array names are transformed into compact deterministic hash vectors rather than sent as raw strings. This representation is sent as JSON to the remote classifier:

```
response = requests.post(
    classifier_endpoint,
    json={"embedding": features.tolist()},
    timeout=config.REQUEST_TIMEOUT_S,
)
```

The classifier result is expected to contain `result`, which is then interpreted as a predicted loop class.

3.5 Loop Class Resolution

The function `_resolve_loop_class(loop_code, pred_class)` combines remote prediction with local safety heuristics. If the classifier predicts a class other than `none`, that class is used. If it predicts `none`, the local logic can still upgrade the loop to:

- `reduction`, when scalar reduction updates are detected.
- `parallel_for`, when consecutive nested `for` loops are detected.

Otherwise the loop remains `none`. This means the module is not purely ML-based; it is a hybrid predictor with deterministic fallbacks for common OpenMP-friendly patterns.

3.6 OpenMP Pragma Generation

`_generate_omp_pragma_with_loop` creates the final annotation. For ordinary parallel loops it emits:

```
#pragma omp parallel for
for (...) {
    ...
}
```

For consecutive nested loops it adds `collapse(depth)`. For scalar addition or multiplication reductions it adds `reduction(+ : var)` or `reduction(* : var)`. The generator therefore currently focuses on OpenMP `parallel for`, `collapse`, and scalar reduction clauses.

3.7 Generator Model Trial and Rule-Based Design Rationale

An earlier generator trial attempted to learn OpenMP pragma generation as a sequence-to-sequence problem. In that formulation, the input sequence contained the loop source code prefixed by a class token such as `[CLS:parallel_for]` or `[CLS:reduction]`, and the target sequence was the expected OpenMP pragma. The implemented model was a custom PyTorch encoder-decoder network:

- **Encoder:** a bidirectional LSTM with embeddings, dropout, packed variable-length input handling, and layer normalization.
- **Decoder:** an LSTM decoder conditioned on Bahdanau additive attention over encoder states.
- **Bridge:** linear projections that map the bidirectional encoder hidden and cell states into the decoder state space.
- **Training:** teacher forcing, cross-entropy loss with padding ignored, Adam optimization, gradient clipping, and a ReduceLROnPlateau scheduler.

The deployed generator service in `models/open_mp_generator/app.py` uses the following configuration: embedding size 128, hidden size 256, three LSTM layers, dropout 0.2, greedy decoding, maximum input length 1500, and a checkpoint saved as `best_model.pth`. The tokenizer is a custom BPE tokenizer rather than a pretrained Hugging Face tokenizer. It is stored in `tokenizer.json`, uses the special tokens `<PAD>`, `<UNK>`, `<SOS>`, and `<EOS>`, and was configured for a target vocabulary of 8000 tokens. The saved tokenizer contains 8002 tokens and 7888 learned BPE merges.

Although this neural generator is expressive, two empirical limitations made a pure learned generator unsuitable for the current system. First, the available training data was small for a code-generation task and contained only a limited effective number of examples for some

pragma classes. In particular, the corpus included about 15k negative examples and a comparatively narrow distribution of positive pragma patterns, which made the sequence model prone to memorization and overfitting. As a result, validation performance did not reliably imply generalization to unseen loop structures, variable names, dependency patterns, or clause combinations.

Second, expanding the corpus with AI-generated examples introduced distribution bias. Synthetic loops and pragmas tend to reflect the style, templates, and regularities of the generating tool. A model trained heavily on such data can learn those artifacts instead of learning robust OpenMP semantics. This lowers real-world generalization even when apparent training or validation error looks acceptable on similarly generated samples.

For these reasons, the implemented generator path was changed to a rule-based approach. OpenMP pragmas are now constructed from explicit static features such as consecutive loop depth and scalar reduction variables. This design is less flexible than open-ended neural generation, but it is more predictable, auditable, and aligned with the current objective: capture the most common and safe pragma forms, including `parallel for`, `collapse(n)`, and scalar `reduction` clauses, without hallucinating unsupported clauses or unbound variables.

3.8 Python-to-C++ Converter

The converter in `py2cpp_converter.py` uses Python’s built-in `ast` module. It does not attempt to be a complete Python compiler. Instead, it emits simple C++ for a restricted subset that is useful for loop experiments. Unsupported syntax raises `ValueError`. The output always contains an `int main()` for top-level statements, while Python function definitions are emitted before `main()` with `auto` return type and `auto` parameters.

Table 4: Supported operations and methods in the Python-to-C++ converter.

Category	Supported form	Generated C++ / Notes
Constants	<code>None</code> , <code>bool</code> , <code>int</code> , <code>float</code> , <code>string</code>	<code>nullptr</code> , <code>true/false</code> , numeric literals, escaped <code>std::string</code> literals.
Names	<code>x</code>	Emitted as the same identifier.
Binary operations	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	Parenthesized infix expressions. <code>**</code> is rejected.
Unary operations	unary <code>+</code> , unary <code>-</code> , <code>not</code>	Mapped to <code>+</code> , <code>-</code> , and <code>!</code> .
Comparisons	<code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	Only one comparison operator is allowed; chained comparisons are rejected.
Boolean operations	<code>and</code> , <code>or</code>	Mapped to <code>&&</code> and <code> </code> .
Function calls	<code>f(a, b)</code>	Emitted as ordinary C++ calls. Keyword arguments are generally unsupported.
Printing	<code>print(a, b)</code>	Mapped to <code>std::cout << a << " " << b << std::endl</code> . Print keywords are rejected.
Attributes	<code>obj.attr</code>	Mapped to <code>obj.attr</code> .
Subscript	<code>arr[i]</code>	Mapped to <code>arr[i]</code> . Slices are rejected.
Assignments	<code>x = expr</code>	First assignment declares inferred type; later assignments reuse the variable. Multi-target and non-name assignments are rejected.

Category	Supported form	Generated C++ / Notes
Augmented assignment	<code>x += y, x -= y, x *= y, x /= y, x %= y</code>	Simple-name targets only. <code>x += 1</code> is emitted as <code>x++</code> .
Return	<code>return</code> or <code>return expr</code>	Emitted directly.
Conditionals	<code>if/else</code>	Standard C++ braces.
For loops	<code>for i in range(...)</code>	Supports <code>range(stop)</code> , <code>range(start, stop)</code> , and <code>range(start, stop, step)</code> with simple name targets.
Functions	<code>def f(a, b): ...</code>	Emitted as <code>auto f(auto a, auto b) {...}</code> . Decorators are rejected.
Type inference	<code>bool, int, float, string</code> constants	Emits <code>bool</code> , <code>int</code> , <code>double</code> , or <code>std::string</code> ; otherwise <code>auto</code> .

3.9 Limitations

The parallelizer is Linux-oriented because it depends on LLVM/Clang development libraries and pybind bindings. It also depends on a remote classifier endpoint unless local integration is added. The Python-to-C++ converter supports only simple syntax, so real Python programs with classes, imports, comprehensions, lists, dictionaries, slices, exception handling, or NumPy calls will not convert directly. The feature extractor is regex-based, so it is fast and transparent but less semantically complete than full dependence analysis.

3.10 How to Use the Parallelizer

After installing LLVM/Clang and building the project bindings, a user can parallelize a C, C++, or supported Python file:

```
from accelera.src.utils.parallelizer import parallelizer

parallelizer.parallelize("examples/loop_example.py")
# Writes examples/parallelized_loop_example.c
```

A simple Python input that the converter can handle is:

```
total = 0
for i in range(0, 1000):
    total += i
print(total)
```

The converter emits C++; the parallelizer extracts the loop and can produce a result similar to:

```
#include <iostream>

int main() {
    int total = 0;
#pragma omp parallel for reduction(+ : total)
    for (int i = 0; i < 1000; i++) {
        total += i;
    }
    (std::cout << total << std::endl);
    return 0;
}
```

4 AutoML Module

The main aim of this module is to automate the machine learning workflow by reducing the manual effort required for data preparation and model development. It is divided into two main submodules:

- **AutoPreprocessing:** Automatically preprocesses tabular, image, and text data, transforming raw data into a format model can work on it for training and inference also generating user-friendly reports.
- **AutoModeling:** Automates model selection, training, and evaluation by identifying suitable machine learning algorithms.

Together, these submodules enable users to build machine learning solutions more quickly and effectively.

4.1 AutoPreprocessing

The aim of Auto Preprocessing to automate common preprocessing steps that are repeatedly performed in machine learning tasks. It also provides a detailed report that includes an overview of the data, all preprocessing steps, and visualizations.

4.1.1 Supported Dataset Types

- Tabular Dataset
- Text Dataset
- Image Dataset

4.1.2 Supported Tasks

- Classification and Regression for tabular dataset
- Classification for text dataset
- Classification and Segmentation for image dataset

4.1.3 Tabular Dataset

The tabular dataset refers to structured data organized in rows and columns, where features can be either categorical or numerical.

Training Preprocessing Steps

1. Load data and user configuration.
2. Get data overview.
3. Drop duplicates.
4. Split dataset.
5. Get training data information.

6. Specify columns to drop.
7. Drop selected columns.
8. Detect the type of each column based on the training data.
9. Generate visualizations of the training data based on column types and task type.
10. Apply feature preprocessing techniques for each column based on its detected type.
11. Apply target preprocessing based on the selected task.
12. Save all files required for preprocessing the test data.
13. Generate Report if user enable report generation.
14. Return `X_train`, `y_train`, `X_val`, `y_val`.

Load data and user configuration: The preprocessing pipeline is initialized through the class constructor. During initialization, all user-defined parameters are assigned and validated to ensure consistency and correctness before executing any preprocessing steps. These validations help prevent invalid configurations such as incorrect problem types, wrong target column name, or invalid hyperparameter values, which could otherwise lead to runtime errors or incorrect model behavior.

Table 5: Classical Training Data Preprocessing Parameters

Parameter	Description
<code>df</code>	The dataframe of the dataset.
<code>target_col</code>	The name of the target column in the dataset.
<code>problem_type</code>	Classification or Regression.
<code>folder_path</code>	Path to store reports, charts, and serialized objects.
<code>val_size</code>	Percentage of data used for validation split.
<code>random_state</code>	To give the same split each run.
<code>cardinality_threshold</code>	Threshold for separating low and high cardinality categorical features.
<code>max_unique_ordinal</code>	Maximum number of unique values to treat integer column as ordinal.
<code>missing_threshold</code>	Drop columns exceeding this missing ratio threshold.
<code>unique_threshold</code>	Drop columns exceeding this uniqueness ratio threshold.
<code>columns_need_to_drop</code>	List of user-specified columns to remove.
<code>is_report</code>	Enables or disables preprocessing report generation.

Data Overview: This step is responsible for generating a comprehensive statistical and structural summary of the dataset before preprocessing begins. It helps generate report.

1. **Capture initial data snapshot:** The first five rows of the original dataset are stored to preserve the original structure before any modifications.
2. **Standardize text data:** All string-based values in the dataset are converted to lowercase, ensuring consistency in categorical values.
3. **Capture transformed snapshot:** After applying lowercase transformation, another preview of the dataset is stored for comparison purposes.
4. **Generate dataset structure information:** to capture column types, non-null counts, and memory usage.
5. **Separate numerical and categorical features:** Columns are divided based on data type into numerical and categorical subsets using `select_dtypes()`.
6. **Compute descriptive statistics:**
 - Numerical features: mean, standard deviation, min, max, quartiles.
 - Categorical features: frequency, unique values, top category.
7. **Compute missing values:** The number of missing values per column is calculated.
8. **Detect duplicate rows:**
 - Total number of duplicate rows is computed.
 - Percentage of duplicated rows is calculated relative to dataset size.
9. **Store dataset shape:** The final dataset dimensions (rows and columns) are recorded.
10. **Save all results into report structure:** All computed statistics, snapshots, and meta-data are stored for later reporting and visualization.

Drop duplicates: Duplicate rows are removed from the dataset to prevent the model from overfitting to repeated examples and to ensure a more reliable evaluation process.

After performing this step, the system reports key statistics to the user, including the total number of duplicate rows detected, the percentage of duplicated data, and the resulting dataset shape after removal. This allows the user to verify that the dataset no longer contains redundant entries.

Split data: Split the data into `X_train`, `X_val`, `y_train`, and `y_val` using the `train_test_split` function from Scikit-learn based on the `test_size` parameter and the `random_state` parameter.

Get training data information: For each column, calculate the required information, such as the percentage of missing values, number of unique values, the mean and median for numerical columns, and the mode value.

Specify Columns to Drop: A column is dropped if:

- It is included in the user-provided `columns_need_to_drop` parameter.
- The percentage of missing values is greater than the user-provided `missing_threshold`.
- The column is a constant value.

Drop Selected Columns: The specified columns are removed from both `X_train` and `X_val`. A preview of the updated datasets is saved to the preprocessing report. Additionally, the column names are serialized and saved as a pickle file to ensure consistency when applying the same preprocessing steps on future data.

Detect the type of each column based on the training data: Each feature is automatically assigned a column type based on its data type and the number of unique values. The detected type determines the preprocessing techniques that will be applied later in the pipeline.

Table 6: Detected column types.

Column type	Detection rule
Binary	Columns containing exactly two unique values or boolean values.
Ordinal	Integer columns with a number of unique values less than or equal to <code>max_unique_ordinal</code> .
Numerical	Integer columns with a number of unique values greater than <code>max_unique_ordinal</code> .
Continuous	Floating-point columns.
Low-Cardinality	Object columns with a number of unique values less than or equal to <code>cardinality_threshold</code> .
Categorical	Object columns with a number of unique values greater than <code>cardinality_threshold</code> .
High-Cardinality	Object columns with a number of unique values greater than <code>cardinality_threshold</code> .
Other	Columns with unsupported data types that are excluded from the preprocessing pipeline.

Generate visualizations of the training data based on column types and task type: The tabular preprocessing report generates a set of graphs for each detected feature type and task type.

- **Binary / categorical features with classification targets:** The report produces these graphs
 - a null-vs-non-null pie chart,
 - a category count plot (Top 5 Categories + Other),
 - a category vs target count plot (Top 5 Categories + Other).
- **Binary / categorical features with regression targets:** The report produces these graphs
 - a null-vs-non-null pie chart,

- a category count plot (Top 5 Categories + Other),
 - a box plot of the continuous target for each category.
- **Ordinal features with classification targets:** The report produces these graphs
 - a null-vs-non-null pie chart,
 - an ordered count plot,
 - an ordered count plot split by class label.
- **Ordinal features with regression targets:** The report produces these graphs
 - a null-vs-non-null pie chart,
 - an ordered count plot,
 - a box plot of the regression target for each ordinal level.
- **Numerical / continuous features with classification targets:** The report produces these graphs
 - a null-vs-non-null pie chart,
 - a histogram with a kernel density estimate,
 - a box plot of the feature grouped by class label.
- **Numerical / continuous features with regression targets:** The report produces these graphs
 - a null-vs-non-null pie chart,
 - a histogram with a kernel density estimate,
 - a scatter plot of feature value against the regression target.
- **Target variable for classification:** The report produces these graphs
 - a null-vs-non-null pie chart,
 - a target class frequency count plot.
- **Target variable for regression:** The report produces these graphs
 - a null-vs-non-null pie chart,
 - a histogram with a kernel density estimate,
 - a box plot of the target distribution.
- **Numeric correlation matrix:** The report generates a heatmap of pairwise correlations across all numeric features.

Table 7: Feature preprocessing applied by column type.

Column type	Applied preprocessing
Numerical / continuous	Imputation with <code>SimpleImputer(strategy="median")</code> , followed by <code>RobustScaler()</code> .
Binary	Imputation with <code>SimpleImputer(strategy="most_frequent")</code> , followed by <code>OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)</code> .
Low-cardinality categorical	Imputation with <code>SimpleImputer(strategy="most_frequent")</code> , followed by <code>BinaryEncoder()</code> .
Frequency-encoded categorical	Imputation with <code>SimpleImputer(strategy="most_frequent")</code> , followed by a frequency encoding transform.
Ordinal	Imputation with <code>SimpleImputer(strategy="most_frequent")</code> , followed by <code>RobustScaler()</code> .

Apply feature preprocessing techniques for each column based on its detected type:

Apply target preprocessing based on the selected task: Target preprocessing changes based on problem type (classification / regression):

- **Classification:** Fill missing target values with the training-mode label, then encode the labels with `LabelEncoder`.
- **Regression:** Fill missing target values with the training median, then standard scale the target using `StandardScaler`.

Save all files required for preprocessing the test data: The preprocessing pipeline persists the objects required for later inference and reuse:

Table 8: Saved preprocessing artifacts for test-time reuse.

Artifact	Purpose
<code>training_preprocessor.pkl</code>	The fitted feature preprocessor.
<code>feature_names.pkl</code>	The output feature names produced by the feature preprocessor.
<code>target_preprocessor.pkl</code>	The fitted target encoder or scaler.
<code>target_info.pkl</code>	Information about the target, including target name, problem type, mode, and median.
<code>bool_type_col.pkl</code>	Boolean columns that are converted to integer values during preprocessing.
<code>data_columns.pkl</code>	The column names of the input <code>DataFrame</code> before preprocessing.
<code>col_drop.pkl</code>	The names of the dropped columns.

Generate Report if user enable report generation: When `is_report` is enabled, the code generates the report from the accumulated `report_data` structure, including the generated graphs, preprocessing summaries, and processed data snapshots.

Testing Preprocessing Steps The testing preprocessing workflow is designed to reuse the exact training preprocessing artifacts and transformations.

1. Verify the required artifact files exist: `target_preprocessor.pkl`, `training_preprocessor.pkl`, `data_columns.pkl`, `col_drop.pkl`, `target_info.pkl`, and `bool_type_col.pkl`.
2. Load the saved dataset columns, dropped columns, target information, boolean-column list, `training_preprocessor`, and `target_preprocessor`.
3. Confirm `target_info` contains a valid `col_name` and `problem_type` (classification or regression).
4. Confirm `target_preprocessor` and `training_preprocessor` is not none.
5. Ensure that the test dataset column order matches the saved training columns. If the target column is missing, testing proceeds in `features_only` mode.
6. Normalize string values in the test dataset by lowercasing them, matching the same text normalization used during training preprocessing.
7. Separate the test DataFrame into `X_test` and `y_test` when the target column is present.
8. Convert boolean columns to integers using the list saved in `bool_type_col.pkl`.
9. Drop the same columns identified during training using `col_drop.pkl`.
10. Apply the saved `training_preprocessor.pkl` transformer to `X_test`.
11. Apply target preprocessing when the target variable is present:
 - For classification, fill missing labels with the training-mode value, then apply the saved label encoder.
 - For regression, fill missing values with the training median, then apply the saved standard scaler.
12. Return the preprocessed test features and labels.

This ensures the test pipeline uses the same feature transformation and target encoding logic as training.

Table 9: Classical Testing Data Preprocessing Parameters

Parameter	Description
<code>df</code>	The dataframe of the test dataset It can contain target column also can not in this case enable <code>feature_only</code> mode.
<code>folder_path</code>	Path to load saved preprocessing artifacts.

4.1.4 Text Dataset

The text dataset consists of unstructured data containing a single text column used for natural language processing tasks. Text preprocessing transforms the raw text column into numerical features suitable for machine learning models.

Table 10: Text Training Preprocessing Parameters

Parameter	Description
df	The input dataframe containing the dataset.
target_col	The name of the target column in the dataset (for supervised classification tasks).
text_col	The name of the column containing raw text data.
folder_path	Path to store reports, preprocessors, and serialized objects. Default: None.
val_size	Fraction of data used for validation split (e.g., 0.2 for 20%). Default: 0.2.
random_state	Random seed to ensure reproducible data splits. Default: 42.
tfidf_max_features	Maximum number of features (vocabulary size) for TF-IDF vectorizer. Default: 500.
tfidf_ngram	N-gram range for TF-IDF vectorizer as tuple (min_n, max_n). Default: (1, 2).
tfidf_max_df	Maximum document frequency threshold for TF-IDF (0.0 to 1.0). Default: 0.85.
tfidf_min_df	Minimum document frequency threshold for TF-IDF (0.0 to 1.0 or integer count). Default: 3.
is_report	Boolean flag to enable or disable preprocessing report generation. Default: True.

Training Preprocessing Parameters

Table 11: Text Training Preprocessing Steps

Step	Description	Output
1	Load data and user configuration	Validated parameters and dataframe
2	Get data overview and compute text statistics	Summary statistics of text data
3	Drop duplicates	Cleaned dataframe without duplicate rows
4	Split dataset into train and validation	<code>X_train y_train X_val y_val</code>
5	Generate visualizations of training data	Graphs saved to folder_path (text distribution, target distribution)
6	Fill null values and normalize text	Text data with nulls replaced and lowercased
7	Tokenize and stem text	Cleaned tokens after stopword removal and stemming
8	Extract text features using TF-IDF vectorizer	Numerical feature matrix from text
9	Apply target preprocessing (label encoding)	Encoded target labels (if applicable)
10	Save preprocessing artifacts	Saved files: TF-IDF model, encoders, vocabulary
11	Generate report (if enabled)	HTML report with statistics and preprocessing details
12	Return results	<code>X_train y_train X_val y_val</code>

Training Preprocessing Steps

Detailed Step Descriptions:

1. **Load data and user configuration:** The text preprocessing pipeline is initialized through the class constructor. During initialization, all user-defined parameters are assigned and validated to ensure consistency and correctness before executing any preprocessing steps.

2. **Text normalization and cleaning:** Text is converted to lowercase, null values are filled with empty strings, and various text preprocessing techniques are applied including removal of special characters, normalization of contractions, and whitespace handling.
3. **Tokenization and stemming:** The custom tokenizer performs word-level tokenization, removes English stopwords (except "not" and "no"), and applies Porter stemming to normalize word forms.
4. **Feature extraction:** A TF-IDF (Term Frequency-Inverse Document Frequency) vectorizer is trained on the preprocessed text to extract numerical features. The vectorizer configuration includes:
 - **max_features:** Maximum vocabulary size (default: 500)
 - **ngram_range:** N-gram range for feature extraction (default: (1, 2))
 - **max_df:** Maximum document frequency threshold to filter common terms (default: 0.85)
 - **min_df:** Minimum document frequency threshold to filter rare terms (default: 3)
5. **Target preprocessing:** Target labels are encoded using LabelEncoder for classification tasks.
6. **Artifact serialization:** All preprocessing components (TF-IDF vectorizer, label encoder, text column name, target column name) are saved as pickle files to ensure consistent test preprocessing.

Table 12: Text Testing Preprocessing Parameters

Parameter	Description
df	The input test dataframe containing the dataset.
folder_path	Path containing saved preprocessing artifacts from training (vocabulary, TF-IDF model, encoders).

Testing Preprocessing Parameters

Table 13: Text Testing Preprocessing Steps

Step	Description	Output
1	Load saved preprocessing artifacts from training	Loaded TF-IDF model, encoders, data info
2	Fill null values in text column	Text data with nulls replaced by empty string
3	Normalize text (lowercase)	Lowercased text data
4	Apply training TF-IDF vectorizer to test data	Numerical feature matrix matching training format
5	Apply target label encoding (if not features-only mode)	Encoded target labels
6	Return results	X_test and y_test (or X_test only)

Testing Preprocessing Steps

Testing Preprocessing Workflow: The text testing preprocessing workflow is designed to reuse the exact training preprocessing artifacts, vocabulary, and feature extraction models. This ensures the test pipeline uses the same text normalization, tokenization, stemming, and TF-IDF feature extraction logic as training. The preprocessing follows these principles:

1. Load all saved artifacts (TF-IDF vectorizer, label encoder, column names) from the training phase.
2. Apply identical text normalization steps as the training phase.
3. Use the fitted TF-IDF vectorizer from training to transform test text into features, ensuring the feature dimensions and vocabulary match exactly.
4. Handle missing target labels gracefully, supporting features-only prediction mode when the target column is absent.

5 Deployment Module

6 Benchmark Platform Module

7 Module-Level View

The five report modules describe different layers of Accelera. The `accelera_pipe` module focuses on graph-based machine learning workflow construction and execution. The code parallelizer focuses on source loop analysis and OpenMP generation. The AutoML, deployment, and benchmark platform sections are reserved for later module descriptions.

Across the implemented sections, a common architectural pattern appears: Python exposes a convenient API and performs validation, while compiled or external services provide lower-level parsing, execution, classification, or infrastructure support.

8 Conclusion

The five-module structure illustrates the central engineering style of Accelera: expose simple Python interfaces while using compiled infrastructure, model services, and supporting platforms for performance-sensitive or operational work. In the current report draft, `accelera_pipe` is documented as a graph-oriented ML workflow system with branching, metric adaptation, path selection, executed graph reuse, and HTML reporting. The code parallelizer is documented as a static-analysis and OpenMP generation tool with a restricted Python converter, feature extraction, classifier-based loop selection, and local heuristic fallbacks. The remaining sections are intentionally left as space for future documentation.